# ASSIGNMENT 6 — QUESTION 2

## Q2 - Task Scheduler: Micro vs Macro Challenge

JavaScript (ES6+) solution with comments

```
// Q2: Event Loop task order
console.log("Start");

setTimeout(() => {
    // setTimeout callbacks are macrotasks (tasks).
    console.log("Timeout callback");
}, 0);

Promise.resolve().then(() => {
    // Promise .then() callbacks are microtasks.
    console.log("Promise callback");
});

console.log("Synchronous log");
console.log("End");

/*
Expected output:
Start
Synchronous log
End
Promise callback
Timeout callback

Explanation:
1. Synchronous statements run first, in source order.
2. After the current call stack becomes empty, JavaScript drains
   the microtask queue (including Promise .then callbacks).
3. The setTimeout callback runs later from the macrotask/task queue.
   A delay of 0 ms does not make it run before microtasks.
*/
```